



Day 5: Advanced CI/CD, AI Debugging & Capstone Project

CI/CD with Jenkins & GenAI Training — Final Day Workshop

ADVANCED WORKSHOP

DAY 5 OF 5

Today's Agenda

A full-day advanced workshop covering the complete modern DevOps engineering stack.



Pipeline Failure Handling

Root causes, logs, and diagnostics



Post-Build Actions

Alerts, archiving, and automation



AI-Powered Troubleshooting

ChatGPT for debugging Jenkins & Docker



Kubernetes Basics + CI/CD

Pods, deployments, rolling updates



Capstone Project

End-to-end pipeline with AI integration



CHAPTER 1

Why Pipelines Fail

Understanding failures is the foundation of resilient DevOps engineering.

Common CI/CD Pipeline Failures

Build Failures

Compilation errors, missing dependencies, or broken Dockerfiles

Test Failures

Unit tests failing due to logic regressions or environment drift

Deployment Failures

Wrong image tags, insufficient permissions, or misconfigured secrets

Infrastructure Issues

Network timeouts, registry unavailability, or out-of-disk errors

Real-World Troubleshooting Scenarios

These failure patterns appear repeatedly in production-grade DevOps environments.

Scenario 1

Docker image pushed to wrong registry tag — deployment pulls stale image silently

Scenario 2

Jenkins agent runs out of disk mid-build; pipeline exits with code 137

Scenario 3

Environment variable missing in production — app crashes only after deploy

Scenario 4

Parallel stage race condition causes intermittent test flakiness

Business Impact of Pipeline Failures

\$300K

Avg hourly downtime cost

For enterprise SaaS outages

23%

Failed deployments

Go undetected for over 1 hour

4x

Recovery time

Without monitoring vs. with observability

Why Monitoring Matters

Pipeline failures without visibility lead to **silent degradation**. Teams need real-time alerting, structured logs, and automated rollback to minimize blast radius.



Every minute of undetected failure compounds downstream impact across dev, QA, and production environments.

Jenkins Console Logs

Logs are your primary debugging instrument — learning to read them fast is a core DevOps skill.



Capture Logs

Identify Stage

Extract Signal

Effective log analysis narrows a failure to a specific stage within seconds, dramatically reducing mean time to resolution (MTTR).

Understanding Jenkins Console Output

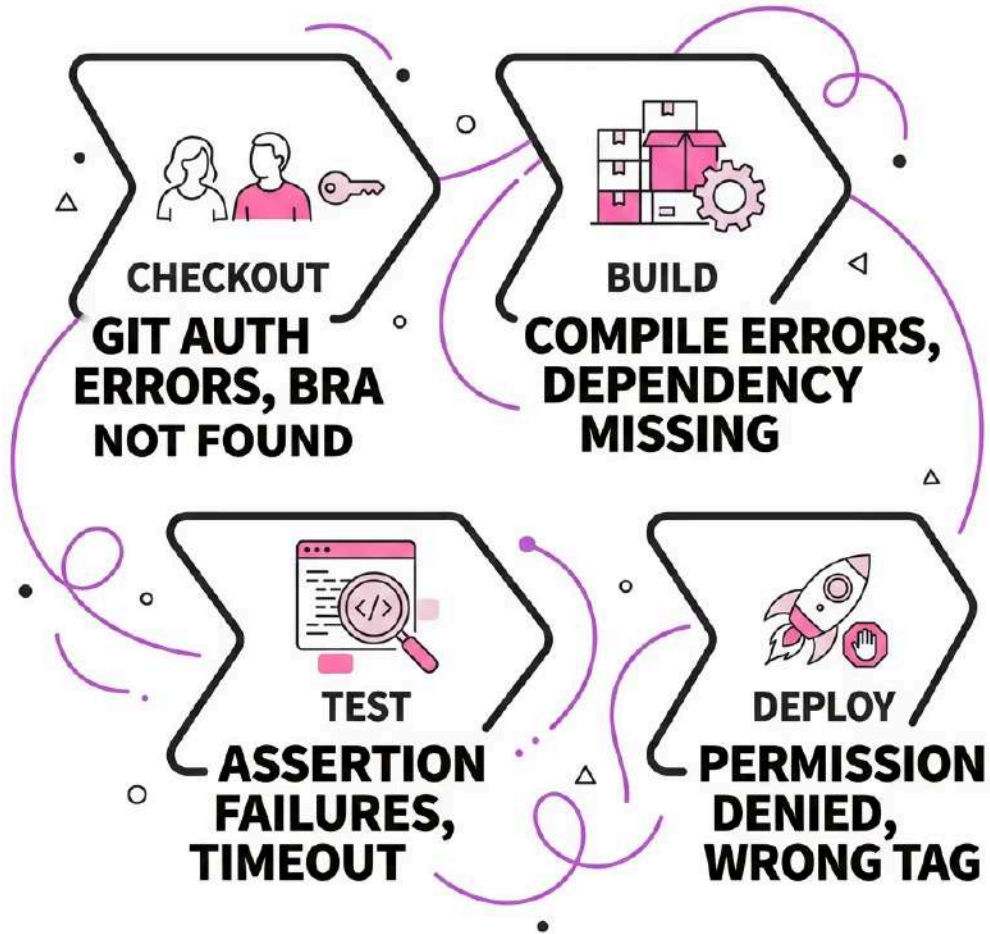
Log Anatomy

Each line is timestamped and stage-scoped. Look for:

- **ERROR** or **FAILED** keywords — immediate failure signals
- **Exit code 1** or **137** — process-level termination
- **Stage name headers** — localize where failure occurred
- **Exception stack traces** — reveal root cause in Java/Groovy

```
[Pipeline] stage
[stage: Build Docker Image]
+ docker build -t myapp:latest .
Step 3/7 : RUN npm install
---> ERROR: npm ERR! code ENOENT
npm ERR! path /app/package.json
[Pipeline] End of Pipeline
FAILED
```


Identifying Failure Stages



Stage Isolation Strategy

Each pipeline stage produces its own log block. Scroll directly to the **first FAILED** stage — upstream stages succeeded and are not the cause.

i Pro tip: Use `pipeline { options { timestamps() } }` to add wall-clock timestamps to every log line for precise failure correlation.

Reading Docker & Jenkins Errors

Docker errors surface inside Jenkins logs — distinguishing them is critical.



Docker Build Error

`COPY failed: file not found` — Dockerfile path mismatch in Jenkins workspace



Registry Auth Failure

`unauthorized: authentication required` — Jenkins credential binding misconfigured



Network Timeout

`dial tcp: i/o timeout` — Agent has no outbound internet or DNS resolution failure



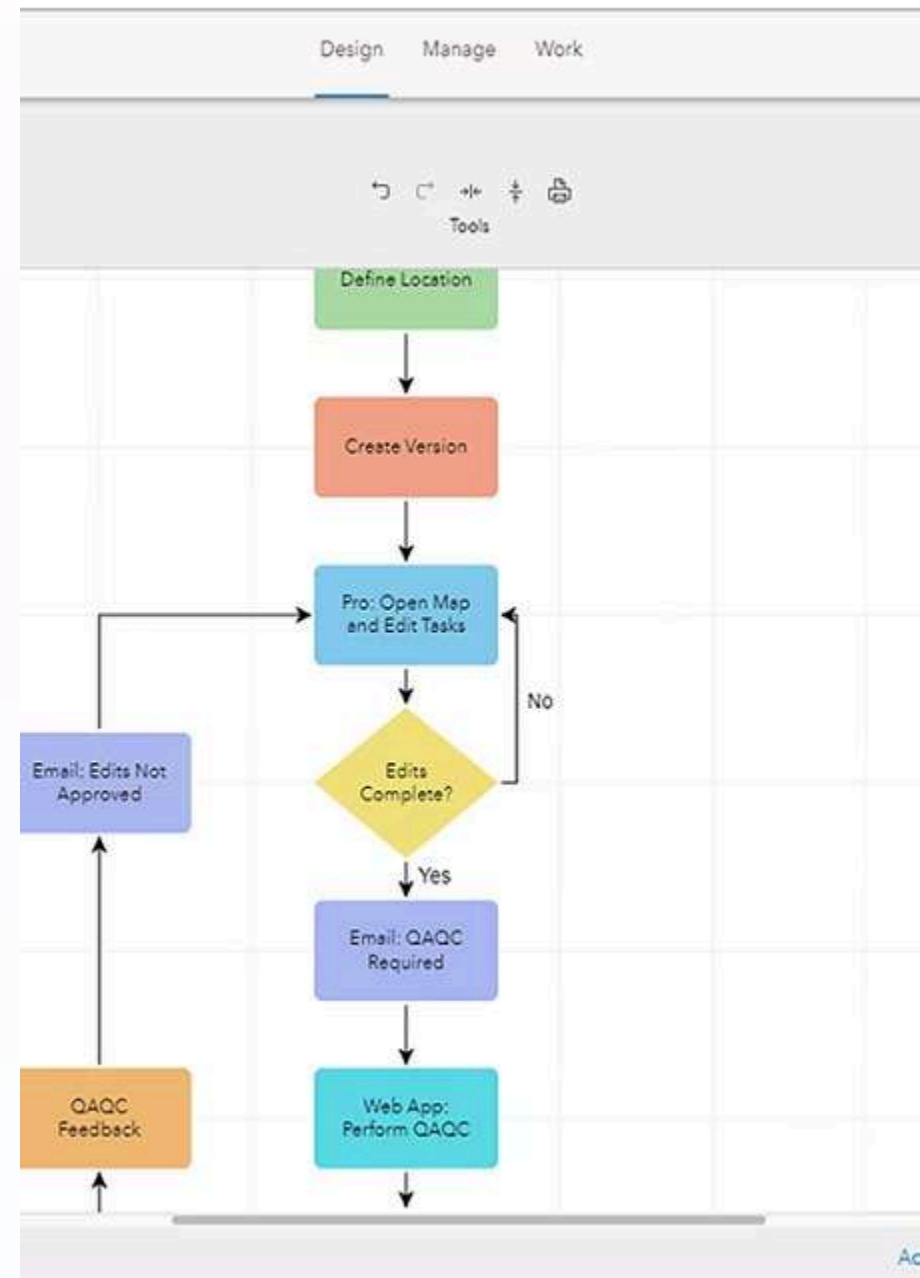
OOM / Disk Full

`exit status 137` — Container killed by kernel OOM killer; increase agent resources

CHAPTER 3

Post-Build Actions

Post-build actions run *after* the pipeline completes — success or failure — and are critical for observability and automated response.



What Are Post-Build Actions?

Declarative Syntax

```
post {  
  always { archiveArtifacts '**/*.log' }  
  success { slackSend 'Build passed'
```

Notifications & Alerts



Slack Integration

Use the Slack Notification Plugin.
Configure webhook URL in Jenkins
credentials and call `slackSend` in
post block.



Email Alerts

Extended Email Plugin supports
HTML templates, CC lists, and
trigger conditions like `FAILURE`,
`FIXED`, or `REGRESSION`.



PagerDuty / Webhook

Use `HttpRequest` step to call any
external webhook — PagerDuty,
Teams, or custom incident
systems.

Log Archiving & Automated Response

Archiving Artifacts

```
post {  
  always {  
    archiveArtifacts(  
      artifacts: 'logs/**/*.*log',  
      allowEmptyArchive: true  
    )  
    junit 'test-results/**/*.*xml'  
  }  
}
```

Automated Troubleshooting

- Archive logs on every run — not just failures
- Trigger a remediation job on failure using `build job`
- Auto-rollback by re-deploying last stable tag
- Send log snippets to AI endpoint for automated root cause

✓ Teams using automated post-build responses reduce MTTR by up to 60% compared to manual triage workflows.

CHAPTER 4

AI-Powered Troubleshooting

Using ChatGPT and LLMs as a real-time debugging co-pilot for Jenkins and Docker failures.

Using ChatGPT for Debugging

The Core Workflow

1 Capture the error

Copy the exact failure message and surrounding log context

2 Add environment context

Mention Jenkins version, Docker version, OS, and pipeline stage

3 Ask for root cause + fix

Request both explanation AND the corrected code or config



Sending Logs to AI: Example Prompts

Well-structured prompts produce actionable, production-ready fixes.

Jenkins Pipeline Error

"My Jenkins pipeline fails at the Docker build stage with:

AI-Generated Fixes in Practice

Before: Broken Stage

```
stage('Build') {  
  steps {  
    sh 'docker build -t app.'  
  }  
}
```

After: AI-Suggested Fix

```
stage('Build') {  
  steps {  
    dir('app') {  
      sh 'docker build -t myapp:${BUILD_NUMBER}.'  
    }  
  }  
}
```

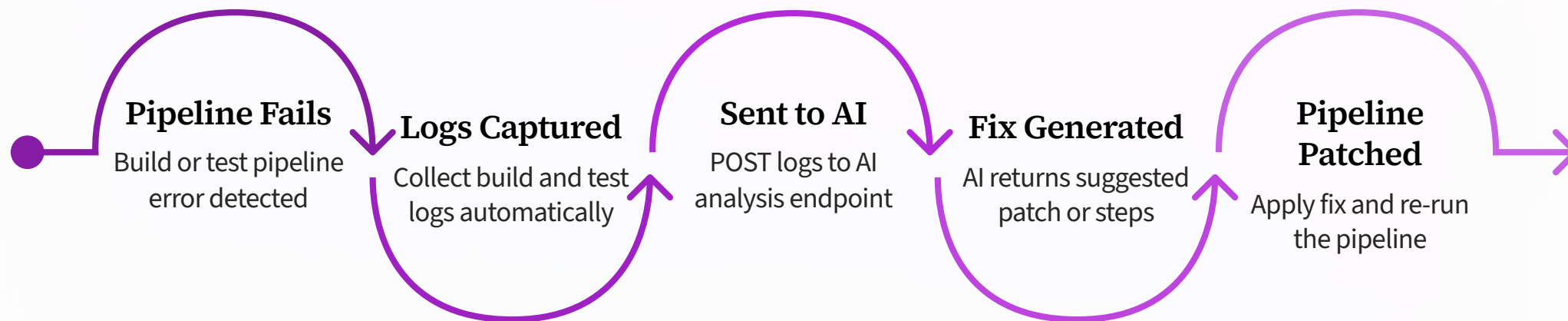
What AI Corrected

- Identified build context mismatch — added `dir()` block
- Added `BUILD_NUMBER` for unique, traceable image tags
- Explained why `.` resolved incorrectly from workspace root

✅ AI explanations accelerate learning — engineers understand *why* the fix works, not just what to change.

Real-World AI DevOps Workflow

The end-to-end loop from pipeline failure to AI-assisted resolution.



This workflow can be fully automated — Jenkins post-build actions capture and POST logs to an AI API endpoint, returning a fix suggestion within seconds.

The Future of AI-Assisted DevOps

AIOps Today

Log analysis, anomaly detection, and incident classification via LLMs

Autonomous Pipelines

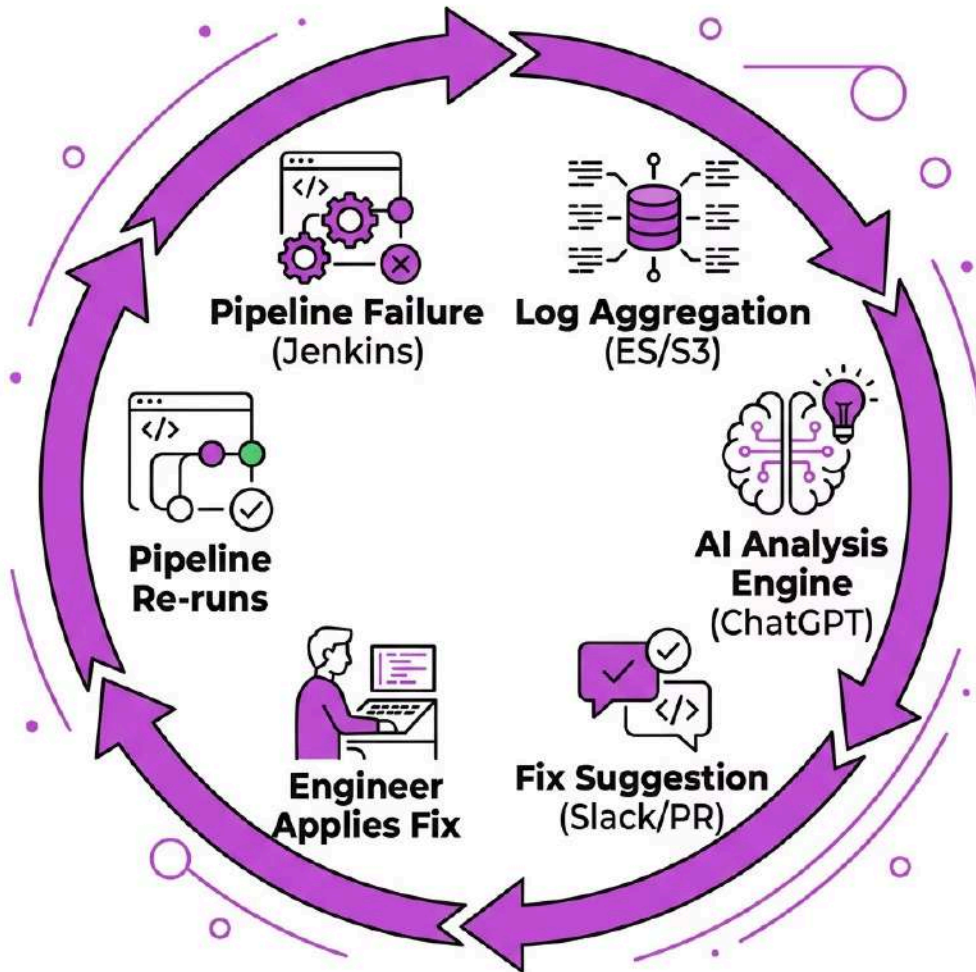
Self-healing pipelines that detect, diagnose, and auto-fix failures

GenAI Code Review

AI-generated PR reviews, Dockerfile optimizations, and security scans inline



Intelligent Troubleshooting: Concept Architecture



Key Integration Points

- **Log Aggregator:** ELK Stack, CloudWatch, or S3 archive
- **AI Engine:** OpenAI API, AWS Bedrock, or self-hosted Ollama
- **Delivery:** Slack message, GitHub PR comment, or JIRA ticket

i Teams at Netflix and LinkedIn use similar closed-loop AI systems to auto-resolve 30–40% of pipeline incidents without human intervention.

CHAPTER 6

Kubernetes Basics

Orchestrating containerized workloads at scale — the natural evolution beyond Docker.



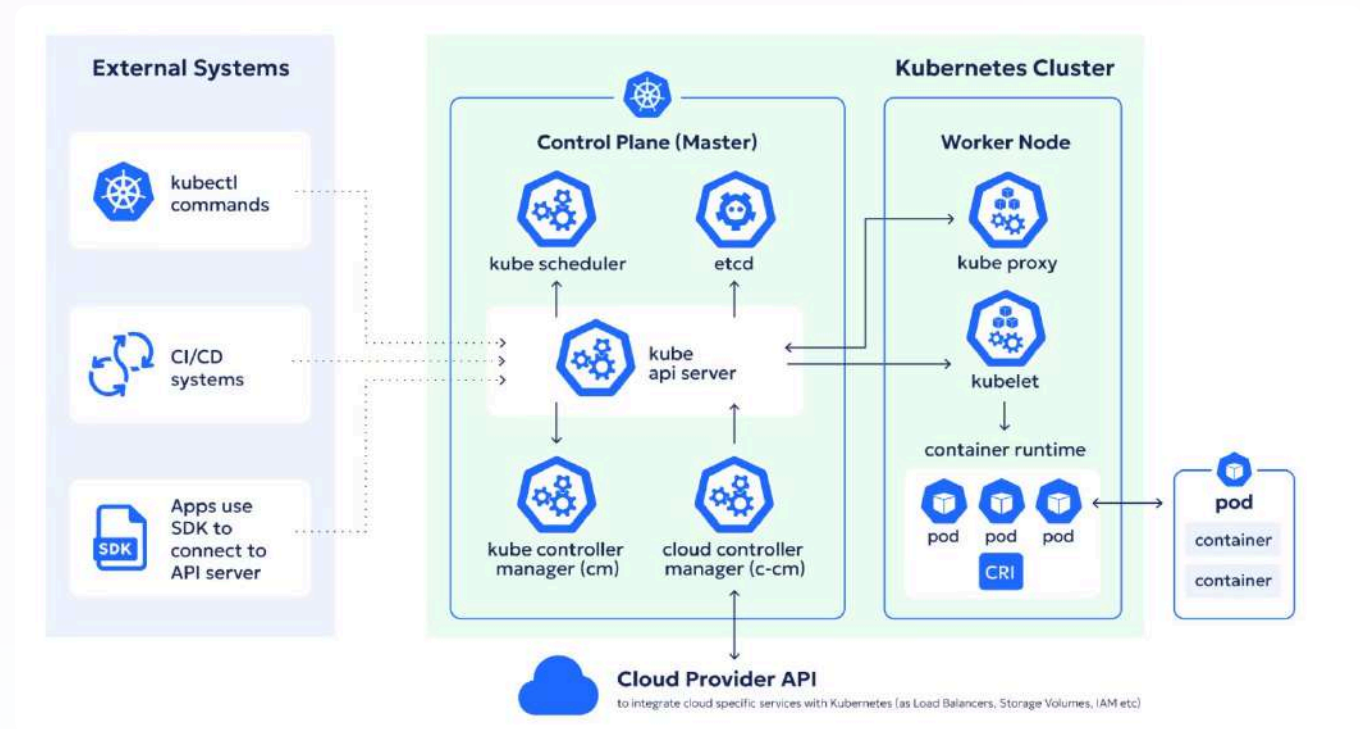
kubernetes

What Is Kubernetes?

The Container Orchestrator

Kubernetes (K8s) automates deployment, scaling, and operation of containerized applications across a **cluster of machines**.

- Originally developed at Google, now CNCF-governed
- Declarative configuration via YAML manifests
- Self-healing: restarts failed containers automatically
- Horizontal scaling: add replicas on demand



Why Kubernetes After Docker?

The Gap Docker Doesn't Fill

Docker runs containers. Kubernetes *manages* them at scale — across multiple hosts, with automated recovery, load balancing, and rolling updates baked in.



Think of Docker as the engine and Kubernetes as the fleet management system.

Pods, Deployments & Services



Pod

The smallest deployable unit. Wraps one or more containers that share network and storage. Ephemeral by design.



Deployment

Manages desired state for a set of Pods. Handles rolling updates, rollbacks, and replica counts declaratively.



Service

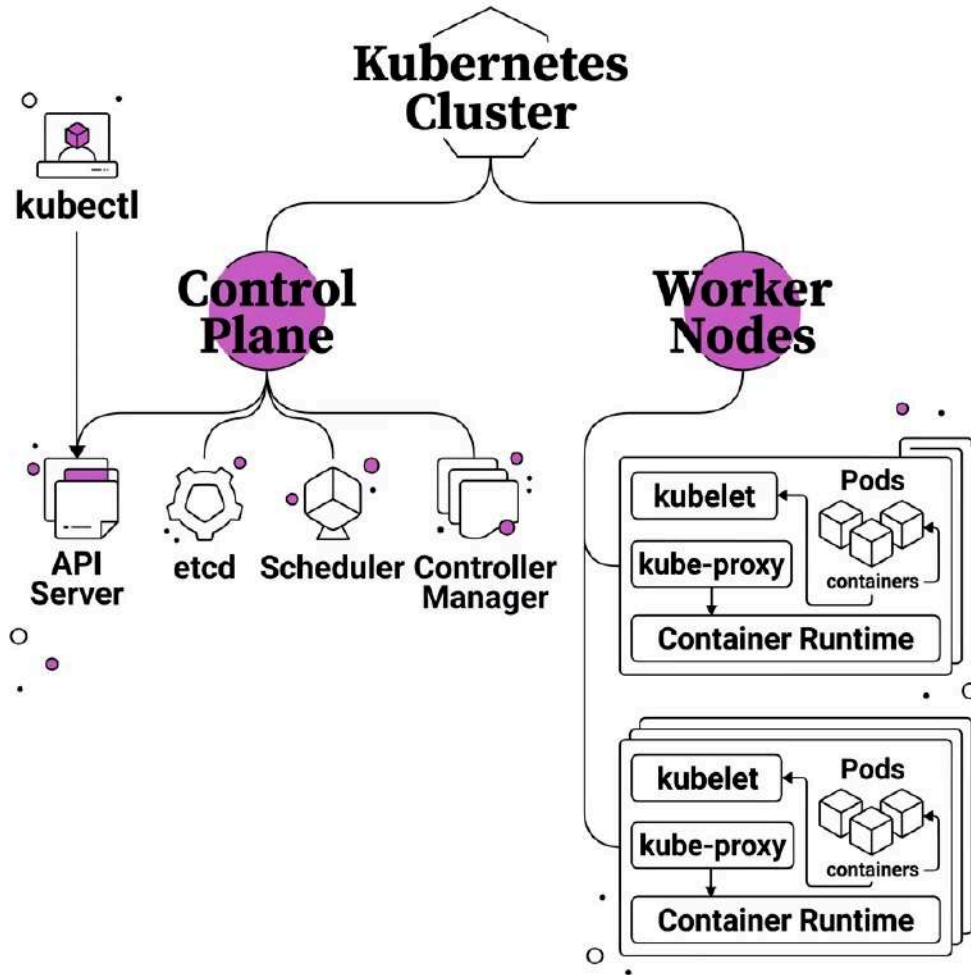
Stable network endpoint for a set of Pods. Provides load balancing and service discovery within the cluster.



Ingress

HTTP routing layer that exposes services externally with path-based and host-based rules. Think reverse proxy.

Kubernetes Architecture

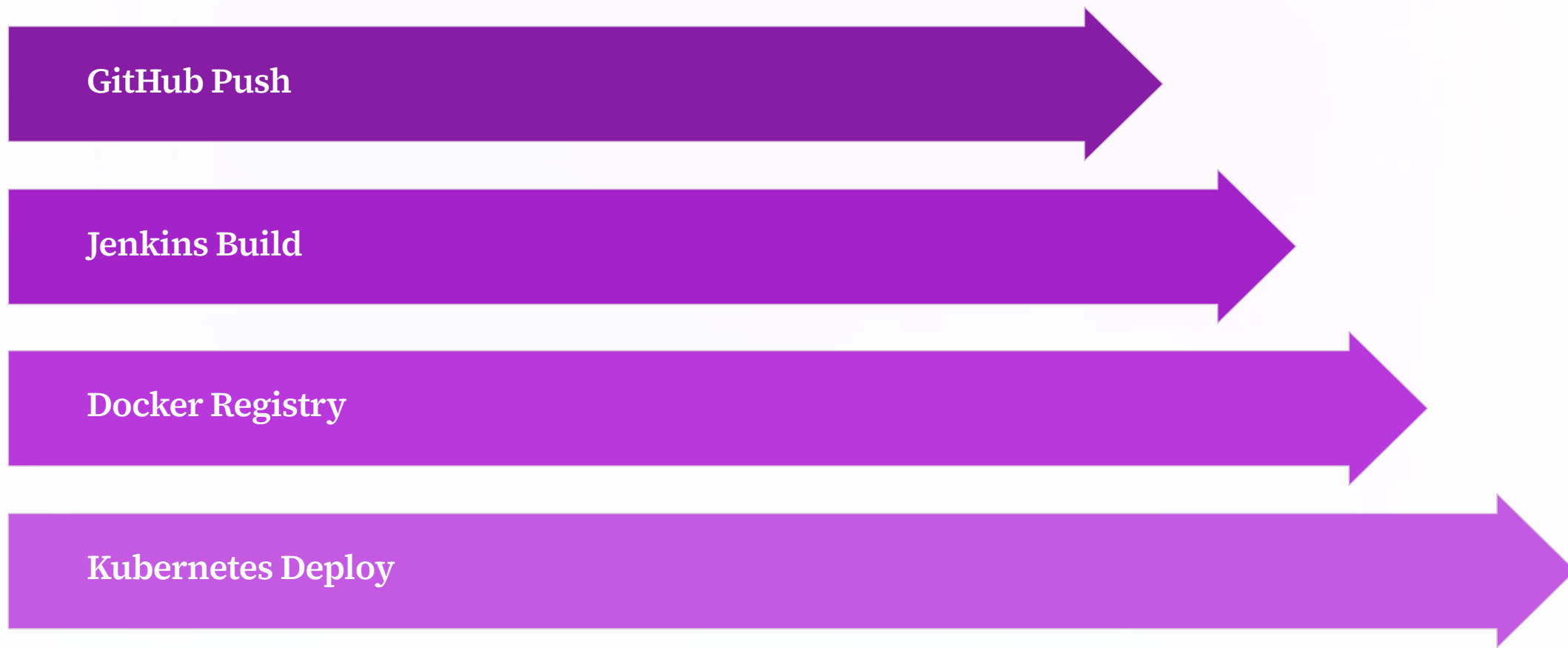


Control Plane vs Worker Nodes

- **API Server:** Single entry point for all cluster operations
- **etcd:** Distributed key-value store — cluster state truth
- **Scheduler:** Assigns Pods to nodes based on resources
- **kubelet:** Node agent that ensures containers run correctly

Kubernetes in CI/CD

Kubernetes becomes the deployment target — Jenkins builds the artifact, Kubernetes runs it at scale.



This pipeline pattern is the industry standard for production Kubernetes deployments, enabling zero-downtime releases with full auditability.

Rolling Updates in Kubernetes

Zero-Downtime Deployment

A rolling update gradually replaces old Pods with new ones, ensuring the application is always available.

```
strategy:  
  type: RollingUpdate  
  rollingUpdate:  
    maxSurge: 1  
    maxUnavailable: 0
```

Key Behaviors

→ New Pod starts

Kubernetes creates replacement Pod before terminating old one

→ Health check passes

Readiness probe confirms new Pod is ready to serve traffic

→ Old Pod terminates

Traffic drains and old container is gracefully stopped

Scaling & Self-Healing

Horizontal Pod Autoscaler

Automatically scales replica count based on CPU/memory thresholds — responds to real traffic spikes

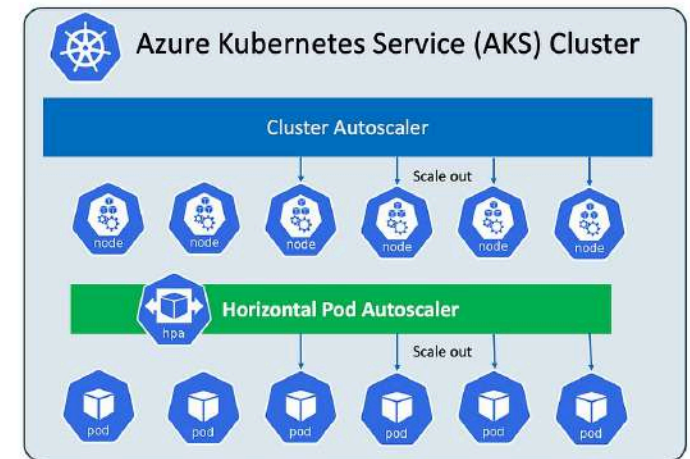
Self-Healing

Controller continuously reconciles desired vs actual state — crashed Pods are restarted automatically, no human action required

CI/CD Trigger

Jenkins applies `kubectl set image` or `kubectl apply` to update the Deployment — Kubernetes handles the rest

Azure AKS – Autoscaling Nodes & Pods



CHAPTER 8

AI for Kubernetes

Use GenAI to generate, explain, and debug Kubernetes YAML manifests — accelerating onboarding and reducing config errors.

Generate Deployment YAML with ChatGPT

Example Prompt

"Generate a Kubernetes Deployment and Service YAML for a Node.js app. Image:

Explaining YAML with AI

When to Use AI for Explanation

- Inherited YAML with no documentation
- Unfamiliar fields like affinity, tolerations, initContainers
- Security context blocks from hardening guides
- Complex multi-container Pod specs

 Prompt pattern: *"Explain this Kubernetes YAML line by line, focusing on what each field controls in production."*

Debugging YAML with AI

Paste a failing manifest and ask:

"My Pod is stuck in

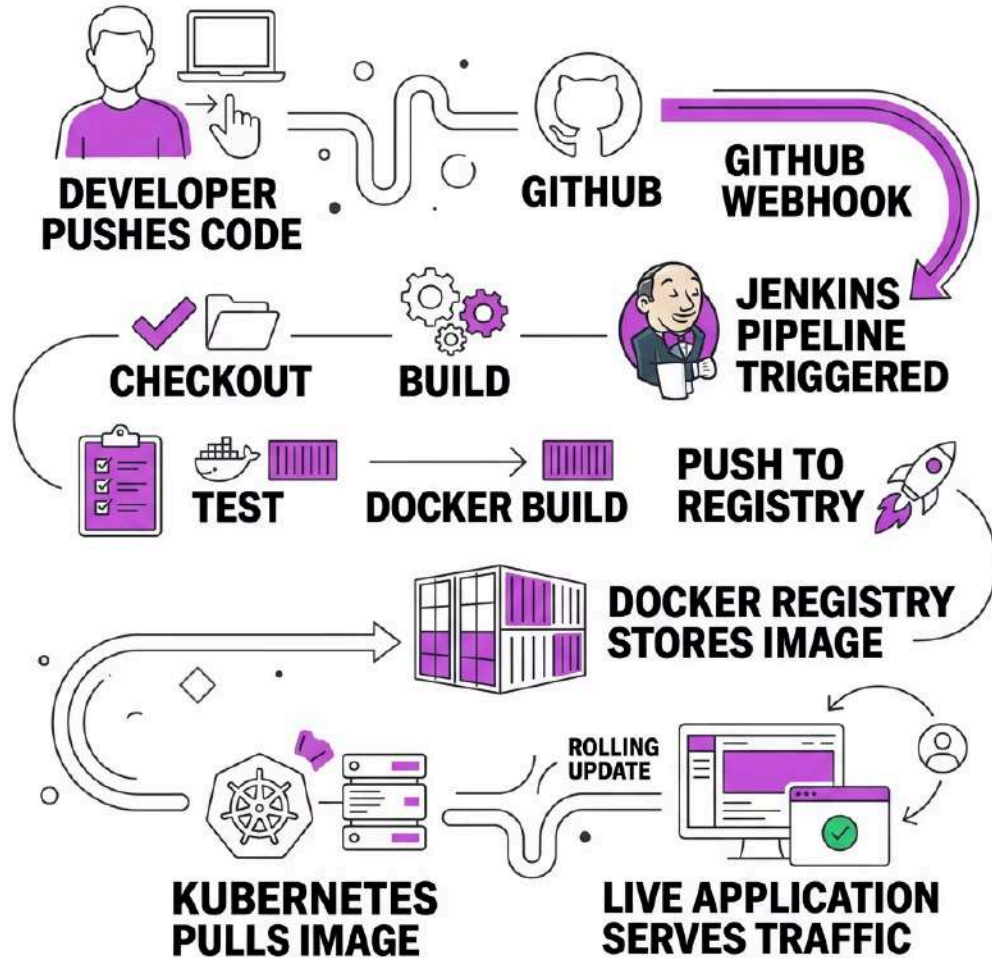


CHAPTER 9

End-to-End CI/CD Pipeline

The complete production-grade DevOps workflow from commit to running container in Kubernetes.

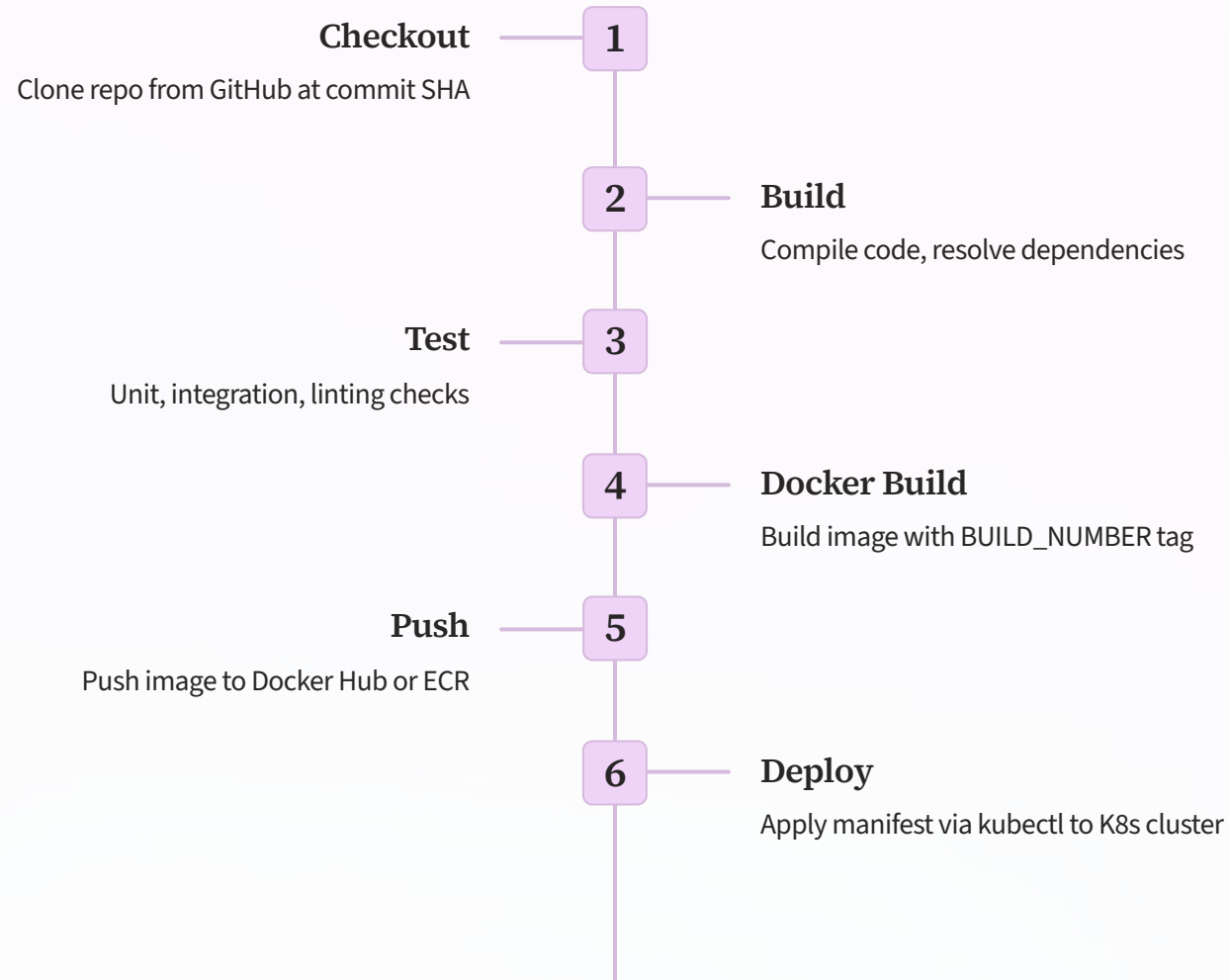
Complete Architecture: GitHub → Kubernetes



Every Component Has a Role

- **GitHub:** Source of truth, triggers automation via webhooks
- **Jenkins:** Orchestrates all build, test, and deploy logic
- **Docker Registry:** Immutable, versioned artifact store
- **Kubernetes:** Runs the artifact at scale with zero downtime

Jenkins Pipeline Stages: Full Reference



Docker Image Build Process

Production Jenkinsfile Snippet

```
stage('Docker Build & Push') {
  steps {
    withCredentials([usernamePassword(
      credentialsId: 'dockerhub-creds',
      usernameVariable: 'DOCKER_USER',
      passwordVariable: 'DOCKER_PASS'
    )]) {
      sh """
        docker build -t $DOCKER_USER/app:$BUILD_NUMBER .
        echo $DOCKER_PASS | docker login -u $DOCKER_USER --
password-stdin
        docker push $DOCKER_USER/app:$BUILD_NUMBER
      """
    }
  }
}
```

Best Practices Enforced

- Credentials never hardcoded — always use Jenkins `withCredentials`
- Image tagged with `BUILD_NUMBER` — fully traceable artifacts
- `--password-stdin` prevents credentials in shell history
- Use multi-stage Dockerfile to minimize image size

Kubernetes Deployment from Jenkins

Deploy Stage

```
stage('Deploy to Kubernetes') {  
  steps {  
    withKubeConfig([credentialsId: 'k8s-config']) {  
      sh """  
        kubectl set image deployment/nodeapp \  
          nodeapp=$DOCKER_USER/app:$BUILD_NUMBER  
        kubectl rollout status deployment/nodeapp  
      """  
    }  
  }  
}
```

What Happens at Deploy Time

- **Image tag updated**
Kubernetes detects spec change and initiates rolling update
- **Rollout monitored**
kubectl rollout status blocks Jenkins until complete
- **Failure triggers rollback**
Add kubectl rollout undo in post failure block

CHAPTER 10

Capstone Project

Build a complete, production-grade CI/CD pipeline — from a GitHub commit to a live Kubernetes deployment — with mandatory AI integration throughout.



Capstone: Mission Brief

What You'll Build



GitHub Repo Setup

Create app repo, configure webhooks to trigger Jenkins on push



Jenkins Declarative Pipeline

Full pipeline: checkout → build → test → Docker build/push → deploy



Docker Image Versioning

Tag with BUILD_NUMBER, push to Docker Hub, verify via registry



Kubernetes Deployment

Apply manifests via kubectl, verify rolling update and Pod health

AI Requirements

- Use ChatGPT to generate your Kubernetes YAML manifest
- Intentionally break a pipeline stage and use AI to diagnose and fix it
- Document your AI prompts and the fixes applied
- Explain AI-suggested changes in your own words in the project write-up

Capstone Evaluation Criteria



Pipeline Completeness

All stages run end-to-end without manual intervention. Post-build notifications configured and firing.



AI Integration Quality

AI used meaningfully for YAML generation and debugging — not just cosmetic use. Prompts documented and results explained.



K8s Deployment Success

Application accessible after rolling update. All Pods in Running state with passing readiness probes.



Debugging Demonstration

Live troubleshooting of an injected failure. Console log analysis + AI-assisted fix applied and verified.

5-Day Skills Summary

From AWS fundamentals to AI-powered Kubernetes deployments — you've built a complete modern DevOps skill set.



Day 5: AI Debugging + Kubernetes + Capstone



Day 4: Docker Integration + AI Debugging Intro



Day 3: Jenkins Jobs, Pipelines, and Stages



Day 2: Git, GitHub, and DevOps Workflows



Day 1: AWS Basics, EC2, and DevOps Concepts

The Modern DevOps + AI Engineering Mindset

Automate Everything

If you're doing it manually twice, it should be a pipeline stage.
Humans for decisions, automation for execution.

Observe, Don't Guess

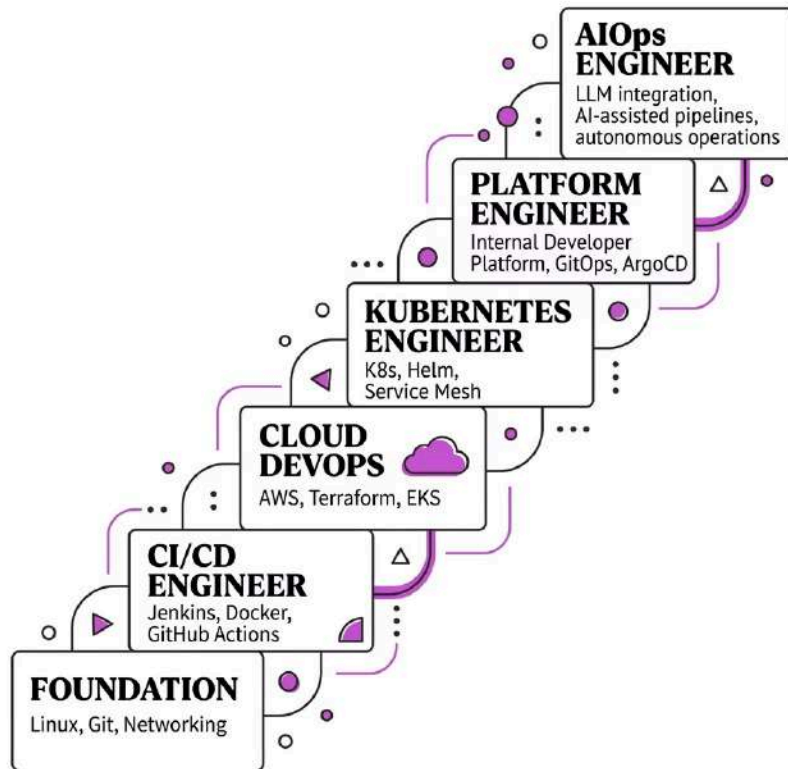
Logs, metrics, and traces are not optional. Build observability in — not as an afterthought.

AI as a Force Multiplier

AI doesn't replace DevOps engineers — it makes them 10x more effective at debugging, generating, and optimizing.



Career Roadmap: DevOps → AIOps



Your Next Steps After This Course

- **Certify:** AWS CCP/SAA, CKA (Kubernetes), Jenkins Engineer
- **Build:** Public GitHub portfolio with CI/CD projects using real apps
- **Explore:** GitHub Actions, ArgoCD GitOps, Terraform IaC
- **Specialize:** Platform Engineering or AI/ML Ops as growth paths

✅ Engineers combining Kubernetes + CI/CD + AI skills are among the most in-demand and highest-compensated in the industry in 2024–2025.

Key Takeaways from Day 5



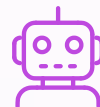
Master Log Analysis

Console logs are your first responder. Read them fast, isolate the failing stage, and extract the error signal.



Kubernetes Is the Standard

Pods, Deployments, and Services are foundational vocabulary. Rolling updates enable zero-downtime production releases.



AI Accelerates Debugging

Structure your prompts with context + error + environment. AI returns fixes, not just explanations.



End-to-End Ownership

A modern DevOps engineer owns the full loop: commit → build → test → deploy → observe → improve.



You've Completed the 5-Day CI/CD Workshop

You now have the hands-on skills, architectural knowledge, and AI toolkit to operate as a **modern DevOps engineer** in real-world production environments.

JENKINS

DOCKER

KUBERNETES

GITHUB

AWS

AI/GENAI